

CSE4121 — Object Oriented Design and Design Patterns

Final Exam Prediction (UPDATED) & Three Model Question Papers

v2 update: the old prediction (2021/2023/2024 papers, CT-1, CT-2, slides, notes) is now MIXED with the new teacher suggestions — the handwritten note and the teal colour-marked sample questions on the last slides of every deck. Where old and new agree, confidence is raised; where the teacher underlined (code smells, MVC, refactoring), the topic jumps to Tier 1.

CHAPTER 1 — Exam Pattern and Source Analysis

1.1 The exam at a glance

Course	CSE4121 — Object Oriented Design and Design Patterns
Time / Full marks	2 hours / 35 marks
Rule	Answer four questions taking two from each section (6 questions printed)
Marks per question	8.75 (splits like 3 + 4.75 + 1, or 4.5 + 4.25, or 5 + 3.75)
Section A (Q1–Q3)	Design principles: OO modelling, CRC, conceptual/technical design, decomposition, inheritance, coupling/cohesion, SoC, code quality
Section B (Q4–Q6)	Design patterns: definitions, categories, Singleton, Factory, Adapter, Facade, Composite, Decorator, Observer, CoR, Command, MVC

1.2 What was analysed (old + new)

Source	What it contributed	Status
Finals 2021 / 2023 / 2024	Fixed skeleton: Q1 modelling+CRC, Q2 decomposition+inheritance, Q3 code-reading; Section B = Singleton/Adapter/Observer/CoR/MVC	old
CT-1 (20 marks)	UML vs IDE; association/aggregation/composition (DUMS); AAS inheritance redesign	old
CT-2 (15 marks)	Observer (complaint status); CoR (Teacher→Chairman→Dean→VC); Adapter (IoT soil sensor)	old
Handwritten suggestion note (7 Oct 2026)	Slide 1: what is design (conceptual/technical), object types, CRC responsibility+collaboration; Slide 2: CT questions; Slide 3: coupling, cohesion, SoC, conceptual integrity (code GIVEN in question — analyse, do not write); behavioural slide: CT-2 questions; pattern slides: GoF; Slide 6 UNDERLINED: code smells, MVC, refactoring	NEW
Teal colour marks on last slides	Exact probable questions marked in Module-1 (both lectures), Module-3 (MVC + smells + principles) and pattern banks — reproduced in Chapter 4	NEW

CHAPTER 2 — Updated Topic Heat-Map and Priority Tiers

2.1 Every topic, every paper, plus the teacher's 2026 marks

Topic	21	23	24	CT	Teacher 2026	NEW prediction
SECTION A						
Object types entity/control/boundary + OO modelling	1a	1a	1a	–	MARKED SI.1	CERTAIN
CRC cards (responsibilities + collaborations)	1c	1b	1b	–	MARKED SI.1	CERTAIN
Conceptual vs technical design / what is design	–	–	–	CT 1	MARKED SI.1	VERY HIGH ↑
Association / aggregation / composition + UML	2a	2a	2a	CT 1	CT listed	CERTAIN
Inheritance design / misuse	–	2b	2c	CT 1	CT listed	VERY HIGH
Coupling & cohesion (code given → analyse)	3b	3a	–	–	MARKED SI.3	VERY HIGH ↑
Separation of Concerns	3c	3b	–	–	MARKED SI.3	VERY HIGH ↑
Conceptual integrity	3a	–	5b	–	MARKED SI.3	VERY HIGH ↑
Code smells + SRP + comments-as-smell	–	–	3	–	MARKED+UNDERLINED	TOP PRIORITY
Refactoring plans / sequences	–	–	3	–	UNDERLINED SI.6	TOP PRIORITY
DRY / Least Knowledge / OCP / Info hiding	–	–	–	–	MARKED SI.6	HIGH ↑
UML vs IDE / design before code	–	–	–	CT 1	CT listed	HIGH
Sequence diagram	2b	–	5a	–	–	HIGH
State diagram / model checking	–	–	5b	–	–	MEDIUM
Trade-offs, entity-boundary extras	–	–	–	–	–	MEDIUM
SECTION B						
What are patterns / GoF / 3 categories	4a	4a, 5a	4a	–	MARKED	CERTAIN
Singleton	4b	5c	4b	–	MARKED	CERTAIN
Adapter	4c	6a	4c, 6b	CT 2	MARKED	CERTAIN
Observer	5b	–	6a	CT 2	CT2 listed	VERY HIGH
Chain of Responsibility	–	6b	–	CT 2	CT2 listed	VERY HIGH
MVC	5a	–	–	–	MARKED+UNDERLINED	VERY HIGH ↑↑
Factory Method	–	–	–	–	MARKED	HIGH ↑
Facade / Facade-vs-Adapter	6	–	–	–	MARKED	HIGH
Composite (file/folder)	–	–	–	–	MARKED	HIGH ↑
Decorator (coffee / notifications)	–	–	–	–	MARKED	HIGH ↑↑
Proxy	–	–	–	–	def. marked	MEDIUM-HIGH
Command / State / Template Method	–	–	–	–	sample qs	MEDIUM-HIGH
Builder / Abstract Factory / Bridge / Filter / Mediator	–	–	–	–	–	LOW-MEDIUM

2.2 Updated priority tiers (old evidence + new marks)

Tier 1 — must know (teacher underlined/marked AND past-paper proven)

Code smells + refactoring + comments-as-smell; MVC (components, SoC, registration scenario); coupling & cohesion; SoC; conceptual integrity; CRC cards; conceptual vs technical design; object types; what-are-patterns + 3 categories; Singleton; Adapter; Observer; Chain of Responsibility.

Tier 2 — very likely

DRY / Least Knowledge / OCP / information hiding; Factory Method; Decorator; Facade (+vs Adapter); Composite; association/aggregation/composition; inheritance redesign; UML-vs-IDE; sequence diagram; Command & State.

Tier 3 — possible

Proxy; Builder; Abstract Factory; Bridge; Filter; Mediator; state diagram & model checking; trade-offs; encapsulation/access modifiers.

2.3 What the new suggestions change

- MVC moves from Tier 3 ("Medium") to Tier 1: the teacher underlined it and colour-marked every MVC sample question.
- Code smells/refactoring move from "High" to TOP: underlined in the note and fully marked on Slide 6 — expect the 100/150-line method, duplicated switch, and a refactoring-plan question.
- Conceptual vs technical design and CRC responsibility/collaboration rise to near-certain (Slide 1 of the note).
- Section A Q3 will GIVE code (flag/switch/large class) — per the note you must analyse it but need NOT write code in the answer.
- Decorator and Composite rise: both colour-marked with coffee and file/folder scenarios.
- Factory Method rises: colour-marked with the TheoryCourse/LabCourse/ProjectCourse design question.
- Behavioural section = CT-2 repeat: Observer, CoR, plus Command/State from the Module-2 sample bank.

CHAPTER 3 — The Teacher's Marked Question Bank (new evidence, verbatim)

These are the exact questions the teacher marked in teal on the final slides, grouped by topic. The Q&A references point to PDF 3 (English Q&A, 58 questions) and to the 1-day PDF "CSE4121_Tomorrow_Exam_Probable_QA.pdf" (T-Q1...T-Q26).

MVC (Slide 6 — underlined, ALL marked)

- What are the three components of the MVC Pattern? State the primary responsibility of each. [T-Q15, PDF3 Q57]
- Explain the responsibilities of Model, View, Controller; how MVC supports separation of concerns. [T-Q15]
- Apply MVC to a university student registration system (identify M/V/C responsibilities). [T-Q16]
- Design an MVC-based Student Management / online course-registration system; how do they communicate? [T-Q16]

Code smells / anti-pattern / refactoring (Slide 6 — underlined, marked)

- Define anti-pattern and code smell; give three examples. Define Large Class, Long Method, Duplicated Code, Primitive Obsession. [T-Q10]
- A method contains 100 (or 150) lines doing validation, database access, calculations and report generation — identify smells + refactoring approach. [T-Q11]
- Same switch statement for customer types in five methods / same student-validation code in five classes — smell, maintenance problems, refactoring. [T-Q12]
- Evaluate whether comments are always beneficial; excessive comments as a design problem. [T-Q13]
- Create a refactoring plan for Large Class + Long Methods + Duplicated Code (sequence of improvements). [T-Q14]
- Evaluate: "A large class is always an example of bad design" / "Design patterns should always be used" — importance of context. [T-Q14]

Design principles (Slide 6 marked + Slide 3 marked)

- What are DRY, Principle of Least Knowledge, Open/Closed, Information Hiding? [T-Q9]
- How DRY improves maintainability; why excessive coupling makes software hard to maintain. [T-Q7/Q9]
- Coupling & cohesion; SoC (SmartPhone); conceptual integrity — code may be GIVEN, analyse only. [T-Q7, T-Q8]

Module-1 patterns (colour-marked on Slide 4 / Module-1 decks)

- What is a design pattern? Three GoF categories; list the seven Module-1 patterns. [T-Q17]
- Singleton: one shared configuration object — implement. [T-Q18]
- Factory Method: TheoryCourse / LabCourse / ProjectCourse design. [T-Q19]
- Adapter: makePayment() vs pay(); IoT soil sensor; problem solved + UML. [T-Q20]
- Facade vs Adapter evaluation; Proxy for access control. [T-Q21]
- Composite for File/Folder hierarchy. [T-Q22]
- Decorator: coffee + milk/sugar/whipped cream; why not subclass combinations. [T-Q23]

Behavioural (Module-2 samples + CT-2 = Slide 5 suggestion)

- Observer: DUMS complaint status notifies citizen, officer, administrator. [T-Q24, CT-2 Q1]
- Chain of Responsibility: Teacher→Chairman→Dean→VC; bank cheque authorisation. [T-Q25, CT-2 Q2]
- Command: remote control / text-editor undo; State vs switch; Template Method. [T-Q26]

CHAPTER 4 — Model Question Paper — SET 3 (new suggestion-based)

University of Rajshahi — Dept. of CSE — B.Sc. (Engg.) Part-IV Odd Semester — MODEL PAPER (SET 3). Course: CSE4121. Time: 2 Hours. Full Marks: 35. Answer any FOUR questions taking TWO from each section.

SECTION A

1. (a) “Design begins before a single line of code.” What is design? Differentiate conceptual design from technical design, naming the tool used at each stage.

[3]

1. (b) Using CRC cards, model the DUMS complaint scenario: show the classes with their RESPONSIBILITIES and COLLABORATIONS (Citizen, Complaint, ComplaintManager, NotificationService).

[4.75]

1. (c) Classify the following AAS classes as entity, control or boundary objects with a reason: Field, SoilSensorReading, IrrigationScheduler, FarmerMobileScreen.

[1]

2. (a) Study the code given below (a DUMS class mixing login, payment, complaint and notification duties with many imports). Evaluate its coupling and cohesion and justify your answer. (Analyse the given code — no new code is required.)

[4.5]

2. (b) Explain Separation of Concerns with the SmartPhone example and define conceptual integrity.

[3.25]

2. (c) State the DRY principle and the Principle of Least Knowledge.

[1]

3. (a) A method contains 150 lines that perform validation, database operations, calculations and report generation. Identify the possible smells and explain how they affect maintainability; suggest a refactoring approach.

[4.5]

3. (b) The same switch statement for customer types appears in five different methods. Identify the code smell and explain why it creates maintenance problems; refactor it.

[3.25]

3. (c) Evaluate: “Comments are always beneficial in software development.”

[1]

SECTION B

4. (a) What is a design pattern? Name the three major GoF categories and list the seven patterns covered in Module 1 with one-line intents.

[2.5]

4. (b) An application requires exactly one shared configuration object throughout the system. Apply the Singleton Pattern with Java code; when is Singleton a bad choice?

[2.5]

4. (c) Design a Factory Method-based system creating TheoryCourse, LabCourse and ProjectCourse; describe classes and relationships.

[3.75]

5. (a) What are the three components of MVC? Explain how MVC achieves separation of concerns in a maintainable web application.

[4.5]

5. (b) Apply MVC to a university student registration system; identify Model, View, Controller and how they communicate.

[4.25]

6. (a) A legacy payment system provides makePayment() while the new application expects pay(); separately, a new IoT soil sensor cannot talk to the existing irrigation controller. Which one pattern solves both? Explain with UML and Java code.

[4.5]

6. (b) A coffee application must dynamically add milk, sugar and whipped cream to a basic coffee object. Apply a suitable pattern and explain why subclassing every combination fails.

[4.25]

CHAPTER 5 — Model Sets 1 & 2 (retained from v1) and Answer Map

Sets 1 and 2 remain fully valid — they are kept in the previous file and summarised here with their answer map. Sit Set 3 first (it matches tomorrow's suggestions), then Set 1, then Set 2.

5.1 Set 3 answer map

Set 3 Q	Topic	1-day PDF	PDF 3
1(a)	Conceptual vs technical design	T-Q1	Q1
1(b)	CRC cards DUMS	T-Q3	Q4
1(c)	Entity/control/boundary	T-Q2	Q7
2(a)	Coupling & cohesion of given code	T-Q7	Q20–22
2(b)	SoC + conceptual integrity	T-Q8	Q23, Q24
2(c)	DRY + Least Knowledge	T-Q9	Q27, Q28
3(a)	150-line method smells + refactoring	T-Q11	Q25
3(b)	Duplicated switch smell	T-Q12	Q27
3(c)	Comments as smell	T-Q13	Q25
4(a)	Patterns + categories + 7 names	T-Q17	Q33, Q34
4(b)	Singleton config	T-Q18	Q35–37
4(c)	Factory Method courses	T-Q19	Q38
5(a)	MVC components + SoC	T-Q15	Q57
5(b)	MVC registration system	T-Q16	Q57
6(a)	Adapter (payment + IoT)	T-Q20	Q41–43
6(b)	Decorator coffee	T-Q23	Q48

5.2 Sets 1 & 2 answer map (unchanged from v1)

Set 1 Q	Topic	PDF 3	Set 2 Q	Topic	PDF 3
1(a)	Design before coding	Q1	1(a)	OO modelling, E/C/B	Q7
1(b)	CRC cards (AAS)	Q5	1(b,c)	Trade-offs; requirements	Q8
1(c)	Design vs architecture	Q6	2(a)	TrafficLight	Q11
2(a)	DUMS decomposition	Q14	2(b)	Drone interfaces	Q19
2(b)	Visibility symbols	Q12	2(c)	Aggregation vs composition	Q15
2(c)	Solar pump refuel	Q18	3(a)	Toll switch, OCP	Q27
3	ServiceManager	Q21	3(b)	Message chain, LoD	Q28
4(a,b)	Patterns; categories	Q33,34	3(c)	Info hiding vs encapsulation	Q12
4(c)	SmsGateway Singleton	Q37	4(a)	Factory	Q38
5(a)	Sequence diagram	Q30	4(b)	Builder	Q40
5(b)	State diagram	Q31	5(a)	Payment Adapter	Q43
6(a)	Observer (weather)	Q51	5(b)	Facade	Q44
6(b)	CoR (escalation)	Q54	6(a)	Composite	Q45
			6(b)	Command	Q55

5.3 Emergency 1-day revision plan (exam tomorrow)

Slot	Work	References
Morning (3 h)	Tier-1 Section A: conceptual/technical design, CRC, object types, coupling/cohesion, SoC, conceptual integrity — write bullet answers	T-Q1–T-Q9
Midday (3 h)	Code smells + refactoring + MVC: memorise the six smell definitions and the 6-step refactoring plan; MVC roles + flow	T-Q10–T-Q16
Afternoon (3 h)	Patterns: Singleton, Factory, Adapter, Decorator, Composite, Facade-vs-Adapter — draw each UML once, write each Java skeleton once	T-Q17–T-Q23
Evening (2.5 h)	Behavioural: Observer, CoR, Command/State drill + pattern-identification list	T-Q24–T-Q26
Night (2 h)	Sit MODEL SET 3 in exam conditions (2 h), then self-mark with the 1-day PDF; skim Set 1 Q3 and Set 2 Q3 code questions	Ch. 4 + Sets 1–2

Full-mark recipe (unchanged from v1): name the pattern + category → map scenario roles → UML class diagram → Java of interface + 1–2 concrete classes + client → 2–3 scenario-tied advantages. For Section A code questions: name the smell/principle first, then justify — and remember the teacher's note that no code needs to be WRITTEN there.